BA Besprechungsnotizen

Bjarne Jehn

April 2026

Besprechung am 08.07.

Nächste Schritte

- gurobi solver probieren - bisher nicht nötig

Fragen

- rshash/test/hashtable.cpp korrigieren: Zeile 72 und 91 jeweils "value" statt "kmer_value" und "value_rev" statt "kmer_value_rev"

Besprechung am 17.06.

Nächste Schritte

- ~~im ilp unterscheiden zwischen variablen und konstanten~~
- ~~erstmal nur ein level modellieren~~
- ~~ilp begründen schriftlich~~

Fragen

-

Besprechung am 10.06.

Nächste Schritte

- linearisiern
- oder einfach mal probieren
- für fixe werte mal durch einen solver laufen lassen und ergebnisse bewerten
- schonmal mit dem schreiben beginnen

- erst danach über code ausdenken
- git!

Fragen

- Sollte man die Space Consumption nicht optimieren sondern eher möglichst genau modellieren, um dann eine obere Grenze wie z.B. 8Gb setzten zu können?

Besprechung am 13.05.

Nächste Schritte

- git zugang

Fragen

- Ist die Annahme, dass die Minimizer-Verteilung in den höheren Leveln ähnlich wie in Level 1 ist allgemein korrekt? - höchst wahrscheinlich schon.
- Wie kann ich festellen, wie viele von 4^m möglichen Minimizern vorkommen? Bzw. ist das überhaupt relevant?
- p_M oder p_K im ILP
- wie kann p_K modeliert werden?

Besprechung am 06.05.

Nächste Schritte

- gleichungen linearisieren, wenn möglich
- p_M^2 und p_M^3 modellieren
- space consumption modellieren

Fragen

- Ist der Zugriff auf R in den Unterschiedlichen Leveln relevant für die Laufzeit? - Denke schon, da dadurch höhere Level "bestraft" werden
- Ist die Zugriffszeit auf R bedeutend abhängig von m und t? - Wahrscheinlich nicht entscheidend aber könnte ebenfalls modelliert werden

Besprechung am 29.04.

Nächste Schritte

- ILP für Laufzeit und Größe überlegen
- tex studio installieren - done
- erstmal nur lookup betrachten + (streaming locate? testen obs funktioniert)
- nach dem timen nochmal über alle positionen iterieren, damit der compiler nichts wegoptimiert
- überprüfen, ob tatsächlicher locate/lookup linear abhängig von minimizer verteilung ist
- in build.cpp gucken für implementierung
- streaming query lookup aus main - queries durch textstellen ersetzten, an denen minimizer mit frequenz i vorkommen
- überlegung: tatsächlich muss ja zunächst im Bitvektor und dann im Text nachgeschaut werden - zeiten variieren je nach system und die parameter können systemabhängig optimiert werden

Fragen

- BA Template? - kommt
- minimizer stats output - geklärt
- Warum kann ich *.../rshash$ .build/source/rshash –help* nicht ausführen aber *.../rshash/build/source$ ./rshash –help* schon? - fehlender backslash im PFad
- Unterscheidet sich mini_stats auf dem dev branch von mini_stats auf dem main branch? - nein
- was wäre eine sinnvolle Projektstruktur, wenn ich selber Code schreiben will? - es müsste möglich sein, programme aus rshash von BA directory aufzurufen Wäre es okay, wenn ich eine Kopie des rshash repositories erstelle und diese in meinem git_lab hochlade? - ja aber nicht nötig
- Fragen zu code:
  - vieles passiert bereits in mini_stats - ist das effizient gelöst? - ja
  - falls ja, kann ich den Code nutzten und anpassen? - ja
  - wäre es sinnvoll die minimizer komprimiert zu speichern? z.B. nur zwei bit pro Base oder gleich als Zahl? - jain, wird durch seqan bib bereits effizient gelöst

– muss ich für jedes m eine Datei mit den minimizern anlegen oder reicht der Arbeitsspeicher, um das ganze in einer Datenstruktur zu behalten? - Arbeitsspreicher sollte vorerst ausreichen. Für sehr große Genome bzw. Sammlungen von vielen großen Genomen wären dateien nötig. Eine Datei wäre vielleicht trotzdem sinnvoll, um später damit zu arbeiten.